

Necessity of Software Evolution

- **Changing requirements:** As an organization's needs change over time, the software must be updated to support new ways of working and improve performance.
- **Environmental changes:** When the working environment or technology changes, software must evolve with updated features to stay compatible and useful.
- **Errors and bugs:** Software defects discovered after deployment must be fixed to improve reliability, stability, and user experience.
- **Security risks:** Outdated software is vulnerable to cyberattacks, making regular updates and security patches essential.
- **New features and performance:** Software must continuously evolve to add new features, improve speed and meet user expectations.

Laws Used for Software Evolution

1. **Law of Continuing Change:** Any software system used in a real-world environment must continuously evolve, or it becomes progressively less useful.
2. **Law of Increasing Complexity:** As an evolving program changes, its structure becomes more complex unless effective efforts are made to avoid this phenomenon.
3. **Law of Conservation of Organization Stability:** The average rate of development of an evolving software system remains relatively constant throughout its lifetime, regardless of the resources allocated.
4. **Law of Conservation of Familiarity:** To maintain familiarity with the software, the amount of change introduced in each release should remain relatively constant. Excessive changes make the software difficult for developers and users to understand.
5. **Law of Self Regulation:** The evolution of software system is self-regulating, meaning its development process follows predictable patterns, and measures such as productivity, growth, and change tend to remain stable over time.
6. **Law of Continuing Growth:** The functionality of software system must continue to grow over time to satisfy users' changing needs and maintain user satisfaction.
7. **Law of Declining Quality:** The quality of software system gradually declines unless it is continuously maintained and updated to adapt to changing user needs and operating environments.
8. **Law of Feedback System:** It depends on continuous feedback from users, developers, and the operating environment. Effective evolution requires using this feedback to guide future changes and improvements.

Advantages

- Keeps software updated to meet changing user and business needs.
- Protects software against new security threats through regular updates.
- Keeps software compatible with new technologies and operating systems.
- Improves usability by adding new features and refining existing ones.
- Keeps software useful and relevant for a longer time.

Challenges

- Requires significant time, effort, and resources.
- Risk of introducing new defects during updates.
- May introduce new vulnerabilities during updates.
- Can cause conflicts with existing systems or software.
- Insufficient testing during evolution may reduce software quality.
- Increasing software complexity makes future maintenance more difficult.

Software Characteristics

```
graph LR; A[Software Characteristics] --> B[Functionality]; A --> C[Efficiency]; A --> D[Reliability]; A --> E[Maintainability]; A --> F[Portability]; A --> G[Usability];
```

Functionality

Efficiency

Reliability

Maintainability

Portability

Usability

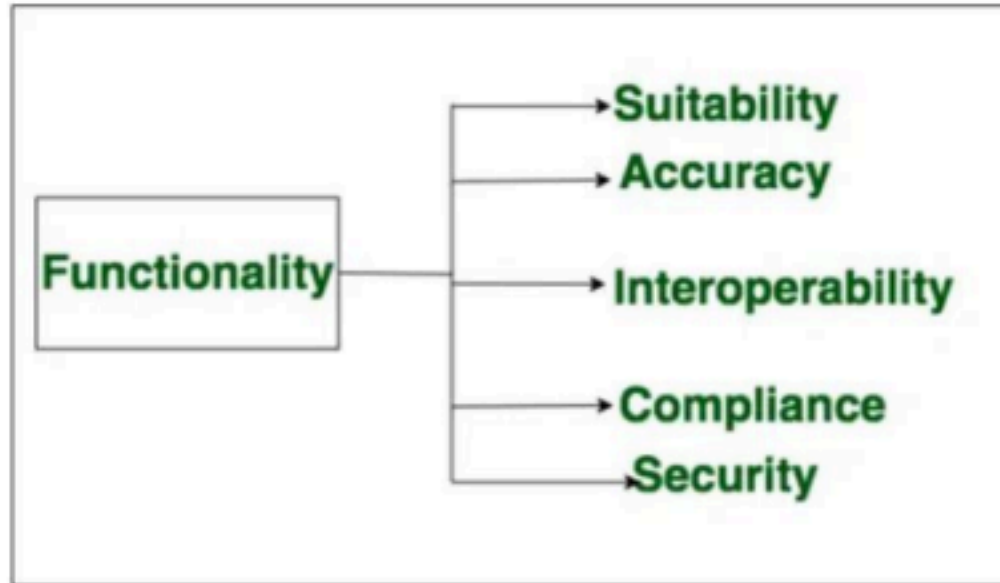
Functionality refers to the set of features and capabilities that a software program or system provides to its users. It is one of the most important characteristics of software, as it determines the usefulness of the software for the intended purpose.

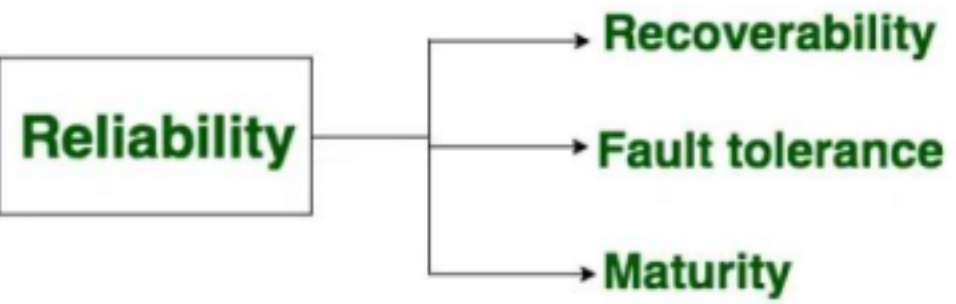
Examples of functionality in software include:

- Data storage and retrieval
- Data processing and manipulation
- User interface and navigation
- Communication and networking
- Security and access control
- Reporting and visualization
- Automation and scripting

The more functionality a software has, the more powerful and versatile it is, but also the more complex it can be. It is important to balance the need for functionality with the need for ease of use, maintainability, and scalability.

Required functions are:





Reliability

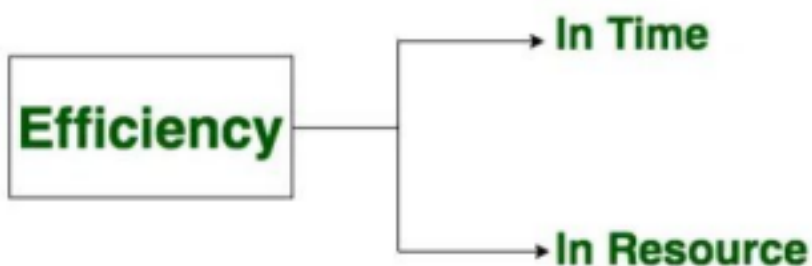
Efficiency:

It refers to the ability of the software to use system resources in the most effective and efficient manner. The software should make effective use of storage space and executive command as per desired timing requirements.

Efficiency is a characteristic of software that refers to its ability to use resources such as memory, processing power, and network bandwidth in an optimal way. High efficiency means that a software program can perform its intended functions quickly and with minimal use of resources, while low efficiency means that a software program may be slow or consume excessive resources.

Examples of factors that can affect the efficiency of the software include:

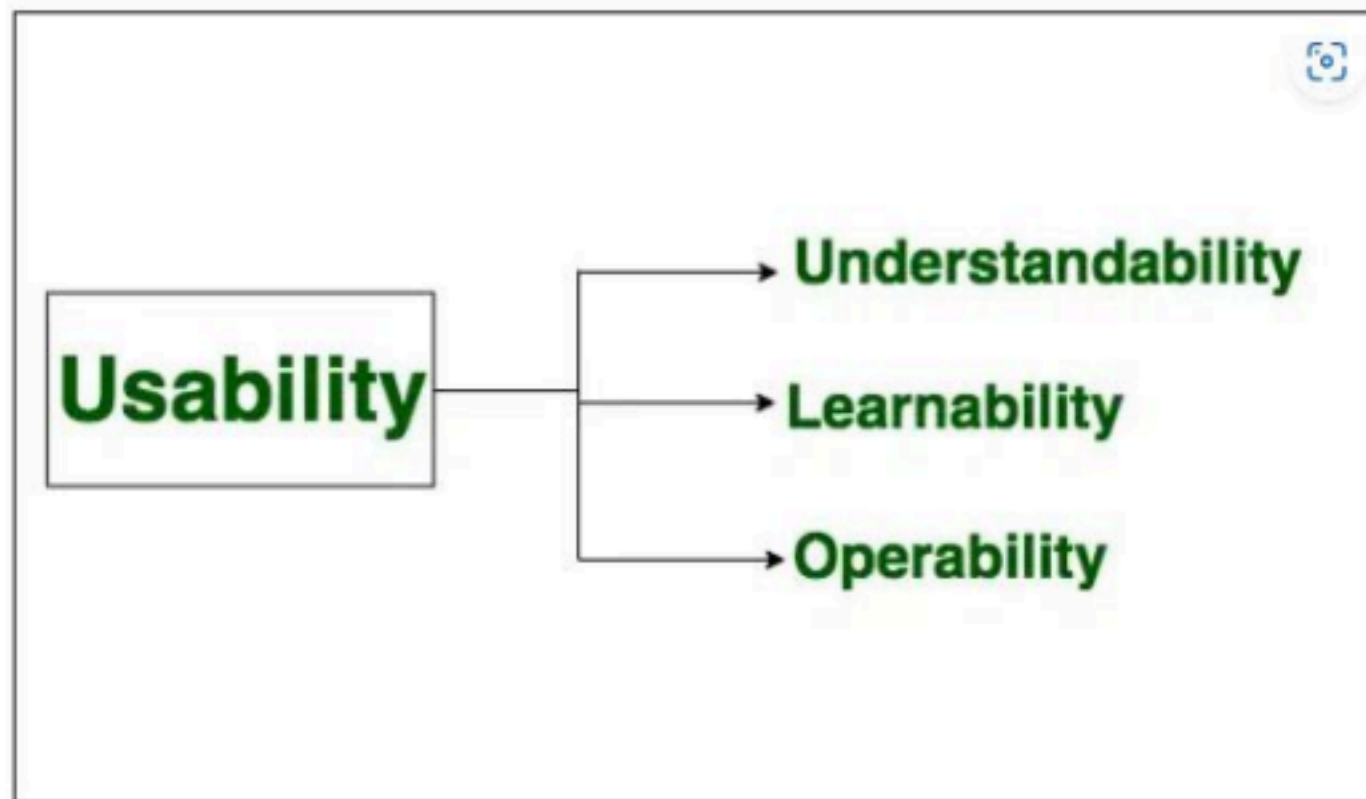
1. Poorly designed algorithms and data structures
2. Inefficient use of memory and processing power
3. High network latency or bandwidth usage
4. Unnecessary processing or computation



Usability:

It refers to the extent to which the software can be used with ease, the amount of effort or time required to learn how to use the software.

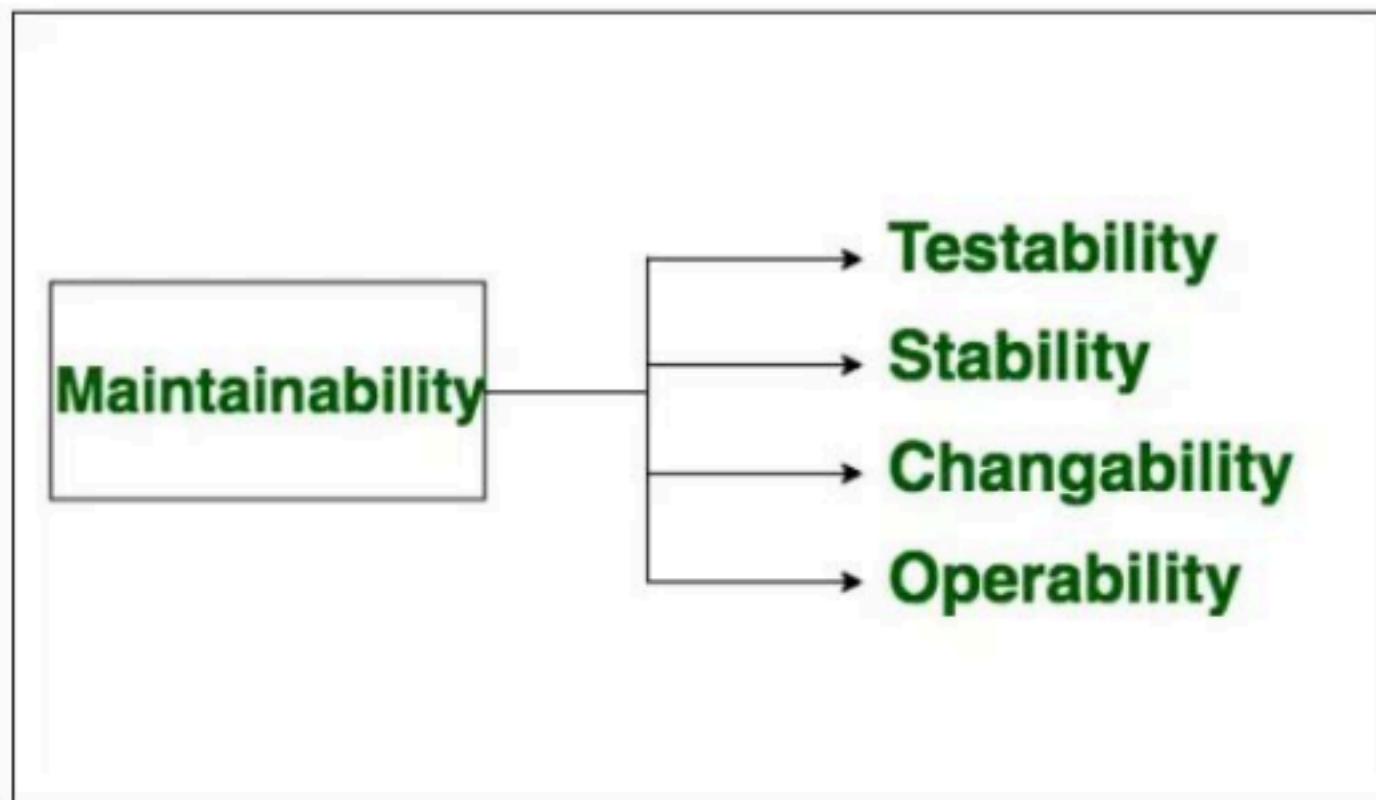
Required functions are:



Maintainability:

It refers to the ease with which modifications can be made in a software system to extend its functionality, improve its performance, or correct errors.

Required functions are:



What is a Software Process Model?

Model

Software Processes is a coherent set of activities for specifying, designing, implementing and testing software systems. A software process model is an abstract representation of a process that presents a description of a process from some particular perspective. There are many different software processes but all involve:

- Specification – defining what the system should do;
- Design and implementation – defining the organization of the system and implementing the system;
- Validation – checking that it does what the customer wants;
- Evolution – changing the system in response to changing customer needs.

Software Process and Software Development Lifecycle Model

One of the basic notions of the software development process is SDLC models which stands for Software Development Life Cycle models. There are many development life cycle models that have been developed in order to achieve different required objectives. The models specify the various stages of the process and the order in which they are carried out. The most used, popular and important SDLC models are given below:

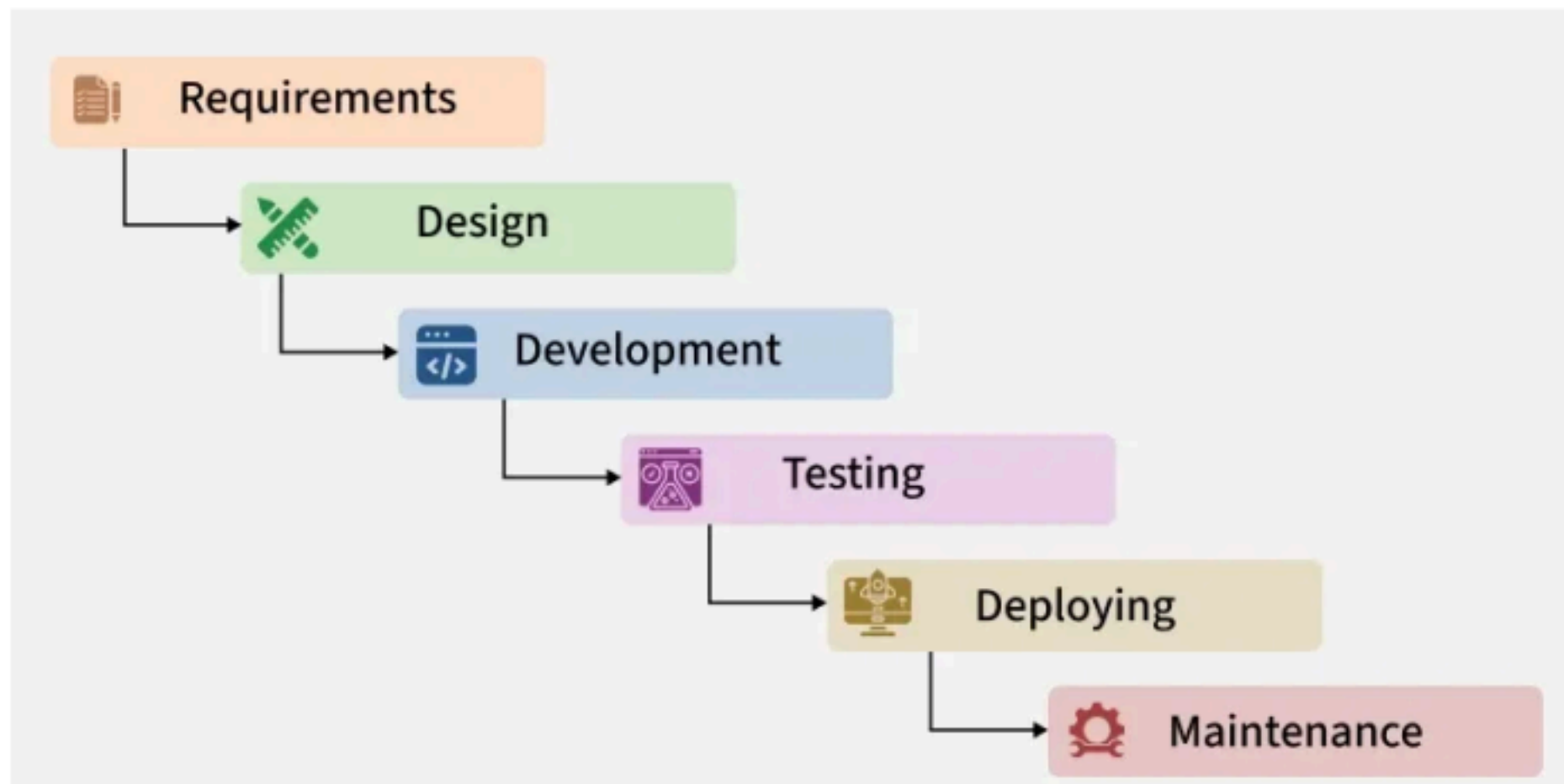
- Waterfall model
- V model
- Incremental model
- RAD model
- Agile model
- Iterative model
- Spiral model
- Prototype model

Waterfall Model - Software Engineering

Last Updated : 15 Jan, 2026



The Waterfall Model is a traditional software development methodology that follows a linear, phase-by-phase approach, where each phase must be completed before moving to the next. It does not allow backtracking and permits only minimal changes once a phase is completed. The model is simple, well-structured, and easy to manage, offering high predictability and clearly defined milestones throughout the development process.



Features of the Waterfall Model

- **Sequential Approach**

Development follows a linear, step-by-step process where each phase is completed before moving to the next.

- **Document-Driven Process**

Detailed documentation is created at every stage to clearly define requirements, design, and progress.

- **Quality Control**

Strong emphasis is placed on verification and testing to ensure each phase meets its defined requirements.

- **Detailed Planning**

The project scope, schedule, and deliverables are carefully planned and monitored throughout the lifecycle.

Phases of Waterfall Model

Classical Waterfall Model divides the life cycle into a set of phases. The development process can be considered as a sequential flow in the waterfall. The different sequential phases of the classical waterfall model are follow:

1. Requirements Analysis and Specification

This phase focuses on clearly understanding and documenting the customer's needs.

- **Requirement Gathering and Analysis:**

Customer requirements are collected and carefully examined to remove errors, confusion, and inconsistencies.

- **Requirement Specification:**

The approved requirements are documented in the [Software Requirement Specification \(SRS\)](#), which serves as a formal agreement between the customer and the development team.

2. Design

In this phase, the requirements from the SRS are converted into a system design that can be implemented in code.

- **High-Level Design (HLD):**

Defines the overall system architecture, major components, and their interactions.

- **Low-Level Design (LLD):**

Provides detailed design of each component, including logic and data flow, to guide developers during coding.

- All designs are documented in the [Software Design Document \(SDD\)](#).

















































































































